



A reusable public transport electronic ticket system with fast validation

Ricard Borges¹ · Francesc Sebé^{1,2} 

Accepted: 24 November 2024 / Published online: 6 February 2025
© The Author(s) 2024

Abstract

This paper presents an electronic ticket system for public transport which provides reusability in the sense that, after the validation of a ticket, a traveler is enabled to perform a journey which can include a limited number of transfers before an expiration time. The proposal has been designed following a privacy-by-design approach. Travelers are only required to identify themselves if requested to prove possession of a valid ticket by an officer. Otherwise, they can travel without being required to disclose their identity (anonymity). Also, the diverse interactions of a traveler with the system cannot be related to them (unlinkability). The security and privacy requirements are achieved by making use of advanced cryptographic techniques. The system has been simulated and proven to provide running times which make it appropriate for a real deployment. When compared to an existing proposal designed for an equivalent scenario, the reduction in the time required for ticket validation is especially remarkable. In effect, all the use cases with real-time constraints (“Ticket validation”, “Get-in”, and “Get-out”) can be run in less than one second.

Keywords Cryptography · E-ticket · Privacy · Reusability

JEL Classification L86 · L91

R. Borges, F. Sebé: these authors contributed equally to this work.

✉ Francesc Sebé
francesc.sebe@udl.cat

Ricard Borges
rborges@matematica.udl.cat

¹ Department of Mathematics, Universitat de Lleida, Av. Jaume II, 69, 25001 Lleida, Catalonia, Spain

² Cybercat: Center for Cybersecurity Research of Catalonia, 25001 Lleida, Catalonia, Spain

1 Introduction

The deployment of advanced Information and communication technology (ICT) solutions in the public transport sector facilitates the implementation of the mobility-as-a-service (MaaS) concept defined by Rantasila (2015) as “*bundling different transport means, public and private, into easy-to-use service to end-customer*”s.

The replacement of classic paper tickets with electronic ones is a key aspect of MaaS. E-tickets allow the implementation of flexible public transport policies such as the possibility to use a single ticket to perform journeys that include transfers (a shift between two or more routes/lines in the same journey) and even make use of diverse means of transport. Moreover, customers can buy tickets everywhere at any moment from their smart devices without having to waste time standing in a queue. The advantages of MaaS are clear, yet they are not exempt of challenges. The existence of standards is a must in scenarios that integrate solutions from different service providers. Covic and Voß (2019) address this issue with the proposal of a standardized process to facilitate interoperability in the scope of smart card-based e-ticket solutions.

An electronic ticket enables its owner to use a service under its terms and conditions. The use of public transport involves the payment of a fare, so electronic tickets for public transport have monetary value. For this reason, a set of security measures must be met in order to avoid frauds (Mut-Puigserver et al. 2012). The most relevant are:

- *Integrity*: any alteration to an e-ticket will be detected and will render it invalid.
- *Authenticity*: only authorized providers can issue e-tickets.
- *Non-repudiation*: customers can prove a ticket was generated by an authorized provider.
- *Non-overspending*: e-tickets cannot be used more times than established.

Regarding the validation of e-tickets, the process can be *online* (validation machines keep a permanent connection to a central server) or *off-line* (validation is performed locally). Note that overspending frauds are a difficult issue for off-line systems.

E-tickets can also be classified as *single use* (just one single journey per ticket) or *reusable* (each ticket can be used up to a maximum number of times (*usage constrained*) and/or until it expires (*temporally bounded*), usually some minutes/hours after its validation).

The management of public transport data in an electronic manner enables the collection of records about the way in which people move in a city. In this way, after performing a proper analysis, the number of vehicles in each route can be optimized and the necessity to establish new routes or upgrade some of the existing ones can be detected. Unfortunately, data mining techniques can also be applied for dishonest purposes. In effect, data about people mobility allow to infer private information such as their place of residence, place of work, vacation period, or hospital attendance, just to mention a few.

Existing legislation, like the European Union General Data Protection Regulation (European Commission 2016), addresses the way in which an organization can process personal data collected from their customers. Unfortunately, legislation is of no use in situations in which data has been stolen as a result of a successful attack against information systems. For this reason, the privacy of customers should also be protected by technological means.

The expected privacy provided by e-tickets depends on the particular means of transport. For instance, plane e-tickets are *identified*, but underground or bus tickets are usually *anonymous*. Anonymity can be provided to different degrees: *fully revocable* (Vives-Guasch et al. 2011, 2012; Aboud et al. 2017) (anonymity can be revoked by a trusted party), *selectively revocable* (Quercia and Hailes 2005) (anonymity can be lifted only in case of misbehavior), or *fully anonymous* (Borges and Seb  2020) (anonymity cannot be revoked in any case). *Unlinkability* is an additional privacy property requiring that the interactions of an anonymous customer with the system cannot be related among them.

In this paper, we present a usage-constrained and temporally bounded reusable electronic ticket system which, like Borges and Seb  (2020), is fully anonymous and all the interactions of a traveler with the system remain unlinkable. While traveling, travelers stand some chance of being requested by an inspector. In such a situation, they are asked to provide a token and prove it is linked both to their identity and to the current public transport trip identifier. The new proposal provides a better performance when compared to that of Borges and Seb  (2020) in two ways: (i) the ticket validation process does not involve the use of costly cut-and-choose proofs for proving the integrity of tokens; (ii) costly zero-knowledge proofs required for proving a token has not expired are used only during face-to-face inspections, instead of each time a traveler begins a journey. As a result, all the time-constrained use cases can be run in less than 1 s. This performance enables the new proposal to be deployed in real scenarios without presenting annoying delays. The new proposal takes an existing single-use e-ticket system and extends it by endowing it with reusability.

The paper has been structured in the following way. Section 1 provides an introduction to the e-ticketing research topic. After that, Sect. 2 reviews related proposals from the scientific literature. Next, Sect. 3 gives an overview of the cryptographic tools employed in this paper. The scenario and security requirements to be satisfied are detailed in Sect. 4, while the new proposal and all the procedures composing it are detailed in Sect. 5. Then, Sect. 6 analyzes the security of the proposal, while its performance is studied in Sect. 7. Finally, Sect. 8 concludes the paper.

2 Related work

The design of secure and privacy-preserving electronic ticket systems is an active area of cryptography research.

Various single-use e-ticket systems have been proposed so far. For instance, Sadeghi et al. (2008) propose an e-ticket system based on the use of radio frequency identification (RFID) technology. The ticket issuer is a trusted party able to identify the travelers. For this reason, we classify this system as non-anonymous since the identity of travelers is kept secret only against external attackers. Vives-Guasch et al. (2011) and Aboud et al. (2017) use pseudonyms to avoid customer profiling. Their anonymity is fully revocable as it can be lifted by court order.

In public transport, granting discounted access to users according to policies related to sensitive attributes such as their age, disability, location, or profession is common. Solutions enabling travelers to benefit from discounts while protecting their privacy have been proposed in e-ticket schemes with attribute-based credentials as it is the case for the single-use ticket systems proposed by Han et al. (2021) or Zhan et al. (2024).

Reusable tickets can be classified into *temporally bounded* (transfers can be made with no limits until expiration) or *usage constrained* (a limited number of transfers is possible). The proposal by Heydt-Benjamin et al. (2006) is an example of the former category in which, after validation, a ticket is valid during a pre-established time interval.

MOTET (Quercia and Hailes 2005) provides usage-constrained reusability. During the issuance of a ticket, which can be used for up to n journeys, an n -element token hash chain $\{chain_\ell\}_{1 \leq \ell \leq n}$ is generated whose tokens satisfy that $chain_{\ell-1} = \mathcal{H}(chain_\ell)$, with $chain_n$ being a random bitstring and $chain_1$ being referred to as the *anchor*. Upon starting the i -th journey, the customer is asked to release $chain_i$. Note that all the journeys made with a given ticket are linkable among them, since all its tokens are linked to the same anchor. The system provides selectively-revocable anonymity, since the identity of the traveler is kept secret unless some token is spent more than once.

Usage-constrained reusability is also provided by Lei et al. (2009) and Vives-Guasch et al. (2012) by means of hash chain techniques. Lei et al. (2009) do not address the anonymity of travelers, while Vives-Guasch et al. (2012) provide fully revocable anonymity since there exists a trusted party with the capability to revoke the anonymity of any e-ticket.

In our opinion, anonymity revocation in public transport tickets only makes sense in scenarios in which travelers are physically prevented from accessing the means of transport unless they provide a ticket which is validated off-line. In effect, when validation is performed online, double-spending attempts are detected immediately in which case the malicious traveler is not granted access, so there is no need to reveal their identity and prosecute them. On the other hand, if an access-blocking system did not exist, malicious travelers would simply get on the transport means without performing any ticket validation. Obviously, misbehaving travelers will refuse to validate a ticket they know will enable their identification.

An alternative anonymity model is that in which tickets remain fully anonymous, but a traveler can prove their ownership if requested by an inspector. In other words, a traveler is the only party able to revoke the anonymity of their tickets. This is the scenario considered by Milutinovic et al. (2015) and Borges and Sebé (2020). Milutinovic et al. (2015) describe a single-use e-ticket system. On the other hand, Borges

and Seb  (2020) present a reusable system in which each ticket can be used up to a maximum number of times before its expiration. That proposal provides unlinkability even among the journeys performed using the tokens generated from a given e-ticket. Unfortunately, the elevated running time of its “Ticket validation” procedure (above 4 s in the fastest setting) introduces a long delay which would be annoying for travelers.

To the best of the authors’ knowledge, Borges and Seb  (2020) is the only temporally bounded and usage-constrained e-ticket system providing also full anonymity, unlinkability and the possibility to accommodate face-to-face random inspections. This paper presents a novel proposal offering the same features. Its contribution is given by the fact that the security requirements are guaranteed by using much more efficient cryptography. As a result, all the time-constrained procedures of the system can be run in less than 1 s on up-to-date devices so that there are no perceptible delays for travelers.

3 Cryptographic background

This section briefly presents the cryptographic tools used by the proposal presented in this paper. The formal notation used along the paper is also introduced here.

3.1 Hash and HMAC digests

A cryptographic one-way hash function (Naor and Yung 1989), $\mathcal{H}$, receives a message M as input and returns a fixed-length digest, denoted $\mathcal{H}(M)$, as output. Hash functions must be *pre-image resistant* and *collision-free*.

A hash-based message authentication code (HMAC) (Bellare et al. 1996) is a keyed one-way hash function. It receives a message M and a key K as input and returns a fixed-length output denoted as $\text{HMAC}_K(M)$. A message and its HMAC digest cannot be related unless the employed key is known.

3.2 Symmetric encryption

A symmetric cipher, like the Advanced Encryption Standard (AES) (Miller et al. 2009), encrypts data by means of an algorithm that takes a cleartext, M , and a secret key, K , as input and produces a ciphertext denoted $\text{AES}_K(M)$ along this paper. Such ciphertext can be decrypted if and only if the key used during its generation is known. In such a case, the cleartext M is obtained.

3.3 ElGamal public key encryption

Elgamal (1985) is a public key cryptosystem whose security holds on the assumed intractability of the discrete logarithm problem. An instance is created by first choosing a large prime number q with $p = 2q + 1$ being also prime. In this way, $\mathbb{Z}_p^*$ is a finite field. Next, a generator g of its order q cyclic subgroup is chosen.

An ElGamal secret key is generated by taking a random integer x in the $[0, q - 1]$ range. The corresponding public key is computed as $y = g^x \pmod{p}$.

Using the digital envelope technique, a message M can be encrypted under public key y by taking a random $r \in [0, q - 1]$ and then computing $k = \mathcal{H}(y^r)$. The resulting ciphertext is composed of $G = g^r$ together with $\text{AES}_k(M)$.

A party in possession of private key x can decrypt the previous ciphertext by computing $k = \mathcal{H}(G^x)$ and using it to decrypt $\text{AES}_k(M)$.

3.4 Digital signatures

A digital signature (Chen 2023) is a cryptographic tool which provides *authentication*, *integrity* and *non-repudiation* to digital communications. A party possessing a private key x can sign a message M by means of an algorithm that takes $\mathcal{H}(M)$ and the private key as input. The result is a signature $\text{Sign}_y(\mathcal{H}(M))$ which can be validated by any party with access to the corresponding public key y . A proper signature validation guarantees that the party who computed it had access to the required private key.

3.5 Blind signatures

A blind signature protocol (Chaum 1984) involves two parties: one of them owns a message M while the other possesses a key pair including public key y . The message owner generates a random *blinding factor*, b , and masks the message hash digest by computing $B = \text{Blind}_y^b(\mathcal{H}(M))$. Then, B is sent to the signer who computes a digital signature on it, namely $R = \text{Sign}_y(B)$ and returns R to the message owner who, by using b , will unblind the result and obtain the desired digital signature $\text{Sign}_y(\mathcal{H}(M))$.

A blind signature protocol guarantees that the signer obtains no information about $\mathcal{H}(M)$ or about the resulting signature $\text{Sign}_y(\mathcal{H}(M))$.

3.6 Partially blind signatures

A partially blind signature scheme (Abe and Okamoto 2000) is an extension of a blind signature scheme which includes some agreed information, *Info*, to the signing process.

Once the message owner and the signer have agreed on *Info*, the signing process includes $\mathcal{H}(\text{Info})$ as part of the blind signature computed over $\mathcal{H}(M)$. The resulting signature will be denoted as $\text{PartialSign}_{\text{Pub}}(\mathcal{H}(M), \mathcal{H}(\text{Info}))$. As with blind signatures, the signer gets no information about $\mathcal{H}(M)$ or about the resulting signature.

3.7 Set membership of committed values

A commitment scheme is a cryptographic primitive which allows to commit to a given value E without revealing it. With c being a random value, $C = \text{Commit}_c(E)$

denotes a commitment on E . By releasing c , any party can check that C is a commitment on E .

Given $C = \text{Commit}_c(E)$ and a set S , Benarroch et al. (2021) propose a technique which allows to prove in zero knowledge that $E \in S$, without disclosing E .

4 System and adversary models

Our proposal endows an existing single-use public transport e-ticket system with reusability. After the validation of a ticket, travelers can start their journey in which a limited number of transfers can be performed before an expiration time. All the single journeys allowed by a given ticket must be made by the same person.

The proposal fits in a proof-of-payment setting (Barabino et al. 2020) in which passengers must have a valid token that allows them to cross a turnstile or gate. Despite this fact, there exists the possibility that some people decide to jump the access barrier or try to travel using tokens belonging to someone else. For this reason, travelers may be subject to random face-to-face inspections.

4.1 System model

In our system, when a traveler is about to start a journey, they validate a ticket of the underlying e-ticket system. For the construction to make sense, the underlying system should be both anonymous and unlinkable. After a proper validation, a set of tokens is created, but only the first of them is *activated*. The traveler will get the rest of the tokens activated sequentially along their journey. These tokens will be presented by the traveler when entering or leaving the system, or if required by an inspector. We consider a scenario composed of the following actors:

1. *Traveler*. Person who uses the public transport system. They carry a mobile device able to run the application used for acquiring and validating e-tickets, and for managing tokens along their journey. This mobile device is approached to entry/exit stations to enter/leave the public transport system. The device is also used to prove token ownership if requested by an inspector.
2. *System server*. This is a central server managed by the service provider. It is accessed by travelers when acquiring or validating e-tickets. It is also accessed by entry/exit stations in order to verify the validity of the tokens presented by travelers and for their activation. Inspectors also access it during an inspection.
3. *Entry/exit stations*. The transport service provider should install a device equipped with proximity reading technology at each entrance/exit point. These devices receive and process the tokens provided by travelers.
 - An entry device grants access to travelers, for instance by lifting a barrier, when a proper “Get-in” token is presented.
 - Exit devices let travelers exit the transport system after receiving an appropriate “Get-out” token.

These devices also participate in the activation of the tokens needed by travelers along their journeys.

4. *Inspector*. Person who roams public transport areas and randomly selects some travelers to check they carry a valid token linked to their identity and to the current public transport trip identifier. The inspector carries a smart device to that end.

4.2 Attacker model

Our system assumes that brute-force attacks against the cryptographic primitives are unfeasible. As to the behavior of a malicious party acting against travelers:

1. An adversary cannot corrupt the app of travelers. Hence, the app runs the protocols as specified and does not leak private information about its users.
2. The system server and the inspectors are honest-but-curious parties. They run the protocols according to their specification, but may collude and share all the information they have access to to try to infer private information about travelers.

On the other hand, travelers may act dishonestly by trying to travel for free or by breaching the terms and conditions of ticket usage (travel after ticket expiration, perform more transfers than allowed, or share tokens).

4.3 Design objectives

The goals of our proposal are the achievement of strong privacy guarantees for travelers and the enforcement of the terms and conditions of ticket usage. More precisely:

1. All the interactions resulting from a traveler using the transport system are anonymous and unlinkable among them. We exclude the particular interactions arising from face-to-face inspections in which travelers are required to tell their identity.
2. In case of inspection, the only information obtained by the system is a Boolean indicating whether the inspected traveler has a valid token enabling them to make the current journey.
3. People traveling without a legitimate token will be caught (and fined) if requested by an inspector.

5 Proposed system

This section provides a detailed description of the proposed system. A summary of the employed notation is given in Table 1.

Table 1 Summary of the notation used in the description of the proposed e-ticket system

Symbol	Description
$Pub^{Transit} / Priv^{Transit}$	Key pair used to authenticate “In-transit” tokens
$Pub^{In} / Priv^{In}$	Key pair used to authenticate “Get-in” tokens
$Pub^{Trav} / Priv^{Trav}$	Key pair used to authenticate “In-travel” tokens
$Pub^{Out} / Priv^{Out}$	Key pair used to authenticate “Get-out” tokens
$Pub^{Id} / Priv^{Id}$	Key pair used to authenticate token sequences
$Pub^{Dep} / Priv^{Dep}$	Key pair used to authenticate “Deposit” tokens
$Pub^{Par} / Priv^{Par}$	Key pair for some partially blind signature scheme
y_t / x_t	Key pair linked to the t -th time slot
ℓ	Number of time slots determining the validity period of e-tickets
M	Number of journeys permitted under a single e-ticket
TT_i	Payload of the i -th token of a sequence
ID_{user}	Digital object with identity information of a traveler
$HMAC_K(D)$	HMAC digest of data D computed under key K
π	$4M$ - element random permutation
$Commit_c(D)$	Commitment to data D under randomness c
$\mathcal{H}(D)$	Hash digest of data D
$Sign_{Type}(\mathcal{H}(D))$	Signature over $\mathcal{H}(D)$ verifiable under public key Pub^{Type}
$B_j = \text{Blind}_{Pub^{Type}}^{b_j}(\mathcal{H}(TT_j))$	Masking of $\mathcal{H}(TT_j)$ under randomness b_j for blind signature computation under the secret key related to Pub^{Type}
$R_j = \text{Sign}_{Type}(B_j)$	To-be-unblinded signature over B_j
G_j	Auxiliar value required for the activation of token TT_j
$AES_K(D)$	AES encryption of data D under key K
ID_{trip}	Current public transport trip identifier

5.1 Time division into slots

Our system divides the time in slots of equal duration (*i.e.* 5 min) which are arranged in a sequential manner. Each slot is identified by means of an integer indicating the number of slots since, for instance, the beginning of year 2023. When a ticket of the underlying system is validated, a set of tokens which will be valid for a previously established number of time slots is generated.

5.2 Sequential use of tokens

A traveler, after validating an e-ticket, can perform a journey that may be composed of several (single) journeys with transfers in between. While traveling from the starting point to their destination, the traveler progressively gets their tokens activated and uses them as described next (Fig. 1 shows it graphically):

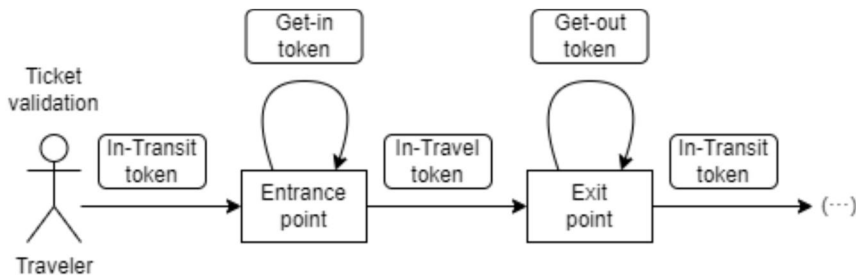


Fig. 1 Sequential use of tokens

1. When a traveler is about to begin a journey, the first action to make is to validate a ticket of the underlying system. This action can be performed a few seconds before reaching the entrance device of the starting point. As a result of this action, the traveler gets an activated “In-transit” token.
2. As soon as the traveler reaches an entrance device, they present the “In-transit” token and obtain an ephemeral activated “Get-in” token.
3. Such “Get-in” token grants access to the public transport system. After presenting it, the traveler is allowed to enter and obtains an activated “In-travel” token which has to be kept while inside the transport system as it will have to be presented if required by an inspector.
4. When the traveler reaches an exit device, they present their “In-travel” token and receive an ephemeral activated “Get-out” token.
5. A “Get-out” token enables the traveler to exit the transport system. If more transfers are still permitted, the traveler gets an activated “In-transit” token which allows them to continue their journey if it has not been concluded yet (in such a case, we jump back to step 2).

Along the paper, the four types of token will be denoted using a subscript or superscript from the set $\{Transit, In, Trav, Out\}$. We also define a successor function, $Succ$, that receives a token type as input and returns the type of its successor. That is, $Succ(Transit) = In$, $Succ(In) = Trav$, $Succ(Trav) = Out$ and $Succ(Out) = Transit$. If the exact type of a token is not relevant, it will simply be denoted as *Type*.

5.3 Overview

The proposal is designed to be attached to some existing single-use e-ticket system. By validating a ticket of the underlying system (Sect. 5.8), a traveler is enabled to generate a sequence of *tokens* (Sect. 5.6) linked to their identity (Sect. 5.7) that are needed for entering (Sect. 5.11) or exiting (Sect. 5.12) the transport system. A token is said to be *activated* when it is accompanied by a digital signature issued by the service provider.

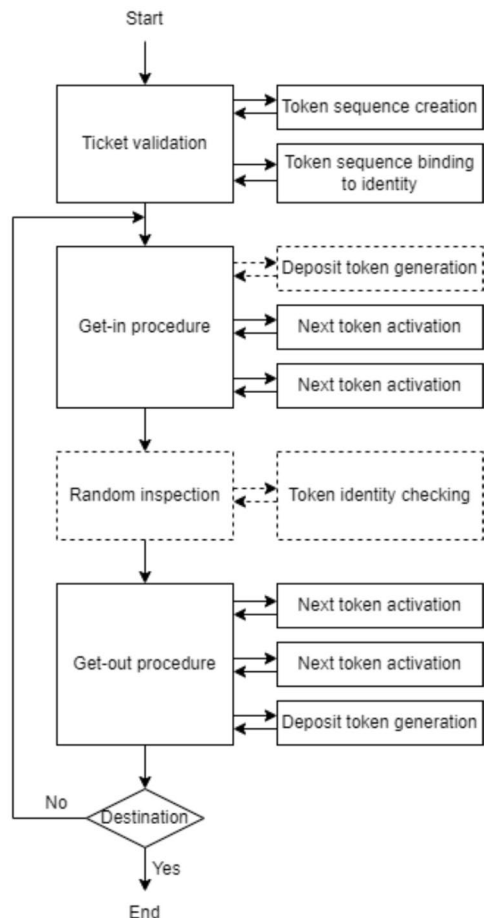
The tokens composing a sequence are generated and stored by the traveler’s app. Then, the app requests the system server to blindly sign each of the tokens. The server acts as required but, as detailed in Sect. 5.8, the resulting signatures,

except for that of the first token, are sent back to the app in encrypted form. In this way, only the first token is activated. The remaining tokens will be progressively activated when the app gets, as detailed in Sect. 5.9, the required decryption key for their signatures, when entering or exiting the public transport system. The system has been designed so that when a token expires, the decryption key for its signature becomes unavailable (Sect. 5.5) so that its signature cannot be retrieved and the token cannot be activated.

In case of inspection, a traveler is required to present an activated token binded to their identity (Sect. 5.13) and to the current public transport trip identifier (Sect. 5.14).

Figure 2 shows the way in which the procedures composing the system are run along a journey.

Fig. 2 Procedure execution during a journey



5.4 Setup

This procedure is run by the service provider before putting the system in operation.

1. The server creates the parameters for an ElGamal cryptosystem (Sect. 3.3): primes p and q together with an order- q element $g \in \mathbb{Z}_p$.
2. The server generates various public-private key pairs for some public key cryptosystem allowing the computation of blind signatures (Sect. 3.5). More precisely,
 - The “In-transit” key pair: $Pub^{Transit} / Priv^{Transit}$.
 - The “Get-in” key pair: $Pub^{In} / Priv^{In}$.
 - The “In-travel” key pair: $Pub^{Trav} / Priv^{Trav}$.
 - The “Get-out” key pair: $Pub^{Out} / Priv^{Out}$.
 - The “Identity” key pair: $Pub^{Id} / Priv^{Id}$.
 - The “Deposit” key pair: $Pub^{Dep} / Priv^{Dep}$.
3. The server generates a key pair for some partially blind signature scheme (Sect. 3.6). The resulting key pair is denoted: $Pub^{Par} / Priv^{Par}$.

5.5 Time slot public keys

Each time slot t is linked to an ElGamal private key $x_t \in [0, q - 1]$ created by the server with the corresponding public key computed as $y_t = g^{x_t} \pmod{p}$.

These keys are managed in a sliding window manner. During time slot t , the server publishes the list of public keys $\mathcal{Y} = \{y_t, y_{t+1}, \dots, y_{t+\ell}\}$. Parameter ℓ has to be chosen so that the public keys in $\mathcal{Y}$ embrace the validity period of tokens created during the t -th time slot. The corresponding secret keys are kept secret by the server.

When time slot t elapses, x_t and y_t are removed from the server memory and from $\mathcal{Y}$, respectively. Then, a new private key $x_{t+\ell+1}$ is created and stored by the server while the corresponding public one $y_{t+\ell+1} = g^{x_{t+\ell+1}} \pmod{p}$ is added to list $\mathcal{Y}$.

5.6 Token sequence creation

Let us assume that system administrators have decided that the validation of a ticket allows to make a journey composed of up to M single journeys.

The (app of the) traveler generates a sequence composed of $4M$ tokens $\{TT_1, TT_2, \dots, TT_{4M}\}$. Each contiguous group of four tokens is employed for performing a single journey. Inside each group, tokens are of “In-transit”, “In”, “In-travel” and “Out” type, respectively.

Each token TT_i is generated by the traveler as a random bitstring. The token length is established to render the probability of random collision negligible (256

bits would be a reasonable option). The particular type of each token will be determined later from the key pair used for computing its signature.

5.7 Token sequence binding to an identity

We denote as ID_{user} a digital object containing information that identifies a given traveler (a picture of them or their passport number).

A token sequence $\{TT_1, TT_2, \dots, TT_{4M}\}$ generated during the t -th time slot (its tokens will expire after slot $t + \ell$) is linked to ID_{user} through the following process run by the traveler:

1. Generate, at random, a set of $4M$ HMAC keys: $\{K_1, K_2, \dots, K_{4M}\}$.
2. Compute a set containing an HMAC digest of each token. More precisely,

$$\{\text{HMAC}_{K_1}(TT_1), \text{HMAC}_{K_2}(TT_2), \dots, \text{HMAC}_{K_{4M}}(TT_{4M})\}.$$

3. Generate a $4M$ -element random permutation π , and generate and store the following bitstring denoted as *Tokens* (note that ID_{user} is appended at the end together with a commitment (c is chosen at random) to the last time slot the tokens in the chain will be valid):

$$\text{Tokens} = \left\{ \text{HMAC}_{K_{\pi(1)}}(TT_{\pi(1)}) || \text{HMAC}_{K_{\pi(2)}}(TT_{\pi(2)}) || \dots || \text{HMAC}_{K_{\pi(4M)}}(TT_{\pi(4M)}) || \right. \\ \left. || ID_{user} || \text{Commit}_c(t + \ell) \right\}.$$

4. Compute the hash digest of the *Tokens* bitstring and ask the server to compute a blind signature on it under the $Pub^{ld}/Priv^{ld}$ key pair. Store the resulting signature $\text{Sign}_{ld}(\mathcal{H}(\text{Tokens}))$.

5.8 Ticket validation

When a traveler is about to begin a journey, they take a ticket of the underlying e-ticket system and validate it against the server. If this validation is successful, the following procedure is run to create the sequence of tokens which will enable the traveler to perform their journey. Let M be the number of single journeys allowed per e-ticket and let t be the index of the current time slot ($y_{t+\ell}$ is the last public key in $\mathcal{Y}$).

1. The traveler generates a sequence of tokens $\{TT_1, TT_2, \dots, TT_{4M}\}$ following the procedure of Sect. 5.6 and binds it to their identity through the procedure of Sect. 5.7.
2. The traveler generates a set of random blinding factors $\{b_1, b_2, \dots, b_{4M}\}$ and creates the set

$$\begin{aligned} & \{B_1, B_2, \dots, B_{4M}\} \\ &= \left\{ \text{Blind}_{Pub^{Transit}}^{b_1}(\mathcal{H}(TT_1)), \text{Blind}_{Pub^{In}}^{b_2}(\mathcal{H}(TT_2)), \right. \\ & \quad \left. \dots, \text{Blind}_{Pub^{Out}}^{b_{4M}}(\mathcal{H}(TT_{4M})) \right\}, \end{aligned}$$

which is sent to the server.

3. The server sets $Type = Transit$
4. The server, for $j = 1$ to $4M$:

(a) Takes the $Priv^{Type}$ secret key and computes

$$R_j = \text{Sign}_{Type}(B_j).$$

(b) Generates $r_j \in [0, q - 1]$ at random, computes $k_j = \mathcal{H}(y_{t+\ell}^{r_j})$, and sets $G_j = g^{r_j}$.

(c) Let $Type = \text{Succ}(Type)$.

5. The server sends the following set to the traveler:

$$\{\{R_1 || G_1\}, \text{AES}_{k_1}(R_2 || G_2), \text{AES}_{k_2}(R_3 || G_3), \dots, \text{AES}_{k_{4M-1}}(R_{4M} || G_{4M})\}.$$

6. The traveler takes R_1 and unblinds it using the blinding factor b_1 to obtain $\text{Sign}_{Transit}(\mathcal{H}(TT_1))$ per result.

After completing this process, the traveler obtains the first activated (signed) “In-transit” token:

$$\{TT_1, \text{Sign}_{Transit}(\mathcal{H}(TT_1))\}.$$

5.9 Next token activation

A traveler who has run the ticket validation process (Sect. 5.8) will sequentially receive the signatures on the tokens of the created sequence along their journey. Assuming the traveler is currently in possession of an activated token $\{TT_i, \text{Sign}_{Type}(\mathcal{H}(TT_i))\}$ and G_i , the next one is obtained as follows:

1. The traveler sends token $\{TT_i, \text{Sign}_{Type}(\mathcal{H}(TT_i))\}$ to the entrance/exit device in front of them.
2. The device verifies the digital signature over $\mathcal{H}(TT_i)$ under public key Pub^{Type} .
3. The entrance device forwards TT_i to the server which checks it has not been presented before. Then, it is stored as an already spent token by the server.
4. The traveler takes a random value $a \in [0, q - 1]$ and sends $G' = G_i^a$ to the entrance device which forwards it to the central server.

5. The central server takes the private keys corresponding to the public keys in $\mathcal{Y}$, namely $\{x_i, \dots, x_{i+\ell}\}$, and computes the set $\{G^{x_i}, \dots, G^{x_{i+\ell}}\}$ which is then transmitted to the traveler via the entrance device.
6. The traveler takes all these received elements and exponentiates them to $a^{-1} \pmod{q}$.
7. If the (non-activated) tokens in possession of the traveler have not expired yet, the hash digest of one of the obtained values corresponds to k_i . The traveler uses it to decrypt $\text{AES}_{k_i}(R_{i+1}, G_{i+1})$.
8. The traveler unblinds R_{i+1} using b_{i+1} obtaining a digital signature of $\mathcal{H}(TT_{i+1})$ under public key $\text{Pub}^{\text{Succ}(\text{Type})}$.

The traveler stores the activated token $\{TT_{i+1}, \text{Sign}_{\text{Succ}(\text{Type})}(\mathcal{H}(TT_{i+1}))\}$ and G_{i+1} .

5.10 Deposit token generation

To ensure that all travelers run the “Get-out” procedure at the end of their journey, we include a token called *deposit token*. Its role in this proposal matches exactly that in Borges and Seb  (2020). Such a token is provided by the traveler when running the “Get-in” procedure and, upon running the “Get-out” one, they receive a new one.

A deposit token is generated free of charge by a traveler when they sign up in the system or after presenting a “get-out” token during the execution of the “Get-out” process. If a traveler has to generate a new one, they will be charged a sum of money.

A deposit token consists of a payload generated at random by the traveler accompanied by its digital signature computed by the server under the *Deposit* key pair, namely $\{\text{deposit}, \text{Sign}_{\text{Dep}}(\mathcal{H}(\text{deposit}))\}$. To prevent tracking, the digital signature is computed through a blind signature protocol.

5.11 Get-in procedure

When a traveler is about to begin a single journey, after validating an e-ticket or after performing a transfer, they are in possession of an “In-transit” token. In this context, ID_{trip} , the current public transport trip identifier, is a value that uniquely identifies each trip carried out by the transport system. It may be derived from the route identifier together with today’s date and a counter.

When reaching a public transport entry point, the traveler approaches their mobile phone to the entrance station. Then, the following steps are taken:

1. The traveler presents an “In-transit” token $\{TT_i, \text{Sign}_{\text{Transit}}(\mathcal{H}(TT_i))\}$ to the entrance station and requests the execution of the procedure in Sect. 5.9. If all the checkings are correct and the token chain has not expired yet, the traveler obtains the next ephemeral signed “Get-in” token $\{TT_{i+1}, \text{Sign}_{\text{In}}(\mathcal{H}(TT_{i+1}))\}$.

2. The traveler runs again the procedure from Sect. 5.9 providing the “Get-in” token $\{TT_{i+1}, \text{Sign}_{In}(\mathcal{H}(TT_{i+1}))\}$ as input. As a result they get the “In-travel” token $\{TT_{i+2}, \text{Sign}_{Trav}(\mathcal{H}(TT_{i+2}))\}$ activated.
3. The traveler presents a “Deposit” token $\{deposit, \text{Sign}_{Dep}(\mathcal{H}(deposit))\}$. If they do not possess one, they will get a new one by running the procedure in Sect. 5.10 during which they will be charged a sum of money. The entrance device checks its digital signature and asks the server to check it has not been presented before. This token is then recorded as already spent.
4. The traveler runs, via the entrance device, an execution of a partially blind signature scheme in which $\mathcal{H}(TT_{i+2})$ is provided as blind input, and ID_{trip} is the agreed information. The resulting signature is stored by the traveler.
5. The entrance station grants the traveler access to the public transport system.

5.12 Get-out procedure

When a traveler is about to conclude a journey they are in possession of an “In-travel” token. When reaching a public transport exit point, they approach their mobile phone to the exit station. Then, the following steps are taken:

1. The traveler presents an “In-travel” token $\{TT_i, \text{Sign}_{Trav}(\mathcal{H}(TT_i))\}$ to the entrance device and requests the execution of the procedure in Sect. 5.9. If all the checkings are correct and the token chain has not expired yet, the traveler obtains the next signed ephemeral “Get-out” $\{TT_{i+1}, \text{Sign}_{Out}(\mathcal{H}(TT_{i+1}))\}$.
2. The traveler runs again the procedure from Sect. 5.9 providing the ephemeral “Get-out” token $\{TT_{i+1}, \text{Sign}_{Out}(\mathcal{H}(TT_{i+1}))\}$. As a result they get the “In-transit” token $\{TT_{i+2}, \text{Sign}_{Transit}(\mathcal{H}(TT_{i+2}))\}$ activated.
3. The traveler generates at random the payload of a “Deposit” token and asks the server to compute a blind signature on it. The resulting signed token is stored by the traveler (Sect. 5.10).
4. The exit station enables the traveler to exit the public transport system.

5.13 Token identity checking

A traveler inside the public transport system should be in possession of an activated “In-travel” token TT_i linked to their identity object ID_{user} . If requested by an inspector during time slot t , the following process will be used to prove TT_i is linked to ID_{user} :

1. The inspector asks the traveler to provide TT_i alongside with the *Tokens* sequence (see Sect. 5.7) and its digital signature $\text{Sign}_{Id}(\mathcal{H}(Tokens))$ which is then validated by the inspector under the Pub^{Id} public key.
2. The inspector takes the ID_{user} object from *Tokens* and checks if the person in front of them corresponds to it.
3. The traveler sends the HMAC key K_i to the inspector (index i is not revealed) and tells the value j satisfying $i = \pi(j)$.

4. The inspector computes $\text{HMAC}_{K_i}(TT_i)$ and checks that the result is in the j -th position of the *Tokens* sequence.
5. The traveler proves in zero knowledge that the *Tokens* sequence has not expired yet. This is done by proving that the commitment at the end of the *Tokens* sequence contains a time slot identifier in set $\{t, \dots, t + \ell\}$ (Sect. 3.7).

5.14 Random inspection

An inspector can ask a traveler inside the public transport system to show they have an activated “In-travel” token linked to their identity object ID_{user} and to the current public transport trip identifier, ID_{trip} :

1. The traveler presents the tuple $\{TT_i, \text{Sign}_{Trav}(\mathcal{H}(TT_i))\}$.
2. The inspector checks $\text{Sign}_{Trav}(\mathcal{H}(TT_i))$ is a valid signature of $\mathcal{H}(TT_i)$ under public key Pub^{Trav} .
3. The inspector asks the system server to check TT_i has not been used before. Then, TT_i is recorded as already used.
4. The inspector asks the traveler to prove their ID_{user} object is linked to TT_i by running the procedure of Sect. 5.13.
5. The inspector asks the traveler to release a partial signature on $\mathcal{H}(TT_i)$ with the current ID_{trip} being the agreed information. This partial signature is validated by the inspector.

After a token inspection, the traveler replaces the presented TT_i with a new one, TT'_i , to prevent tracking:

1. The traveler generates a new random token TT'_i and asks the server to compute a blind signature on it. The resulting $\text{Sign}_{Trav}(\mathcal{H}(TT'_i))$ is stored by the traveler.
2. The traveler generates a random HMAC key K'_i , computes $\text{HMAC}_{K'_i}(TT'_i)$ and overwrites the j -th position (Sect. 5.13) of their *Tokens* sequence with this result.
3. The updated *Tokens* sequence is sent to the inspector who checks that only its j -th position has been modified. Then, the inspector computes a digital signature (under the Id key pair) on it and sends it to the traveler who stores it.
4. The traveler runs, via the inspector, an execution of a partially blind signature scheme against the server in which $\mathcal{H}(TT'_i)$ is provided as blind input, and ID_{trip} is the agreed information. The resulting signature is stored by the traveler.

6 Security analysis

The following two subsections analyze the security of the system in terms of security for the traveler against an automatic tracking of its traveling habits (Sect. 6.1) and security for the system against travelers aiming to violate the terms and conditions of the e-ticket system (Sect. 6.2). Each security topic is addressed by means of a set of lemmas proving specific statements and a conclusion theorem.

6.1 Privacy analysis

This section addresses how the system prevents an automatic tracking of travelers.

Lemma 1 *During an execution of the “Ticket validation” procedure, the server gets no information about the generated token sequence.*

Proof During this process, the traveler generates at random each of the tokens composing the sequence by running the “Token sequence creation” process. Since tokens just contain random data, they cannot be identified from their payload. These tokens are kept by the traveler.

Next, tokens are linked to the traveler identity through the “Token sequence binding to an identity” procedure. The server participation in this process is reduced to the computation of a blind signature over the sequence called *Tokens*. A blind signature protocol guarantees that the server gets no information about the signed data (Sect. 3.5).

Finally, as part of the “Ticket validation” process, the server computes a blind signature over each of the created tokens. The server neither gets any information about them now. $\square$

Lemma 2 *The tokens of a sequence cannot be related among them.*

Proof Since the payload of a token is random and its signature is only linked to its payload, neither the payload nor its signature provides any information enabling its linkage to other tokens. From Lemma 1, the server cannot relate tokens to the moment they were generated. In this way, tokens cannot be linked, neither from their payload nor from their generation time. $\square$

Lemma 3 *During an execution of the “Next token activation” process, the server gets no information about the newly activated token.*

Proof During this process, the server receives the current token, TT_i , which is recorded as already spent. From Lemma 2, tokens cannot be related among them. So the server has no way to relate TT_i to any other token deposited in the past nor in the future.

The server also receives G' (at step 4) which is a masked version of G_i , namely $G' = G_i^a$ for some random integer a . From the way it is generated, G' is a random element from the order- q subgroup of $\mathbb{Z}_p$ with no relation to any piece of data of the system. $\square$

Lemma 4 *After a “Token inspection”, the execution of the “Get-in” and “Get-out” procedures of the inspected journey cannot be determined.*

Proof During an inspection the traveler presents an activated “In-travel” token, TT_i . From Lemmas 2 and 3, this token cannot be linked to the “In-transit” nor the “Get-in” tokens presented when the traveler got into the public transport system.

Next, the traveler presents the *Tokens* sequence TT_i belongs to together with the HMAC key K_i needed to compute the HMAC of TT_i included in *Tokens*. Other tokens included in *Tokens* cannot be determined by the server because the needed HMAC keys are not provided by the traveler.

After a successful inspection, the traveler generates a new token TT'_i which is blindly signed by the server. Then, a new *Tokens'* sequence is created in which the HMAC of TT'_i computed under a new key K'_i replaces the position of TT_i in the original *Tokens* sequence. During this process, the server has access to *Tokens'* but since K'_i is not revealed, it gets no information about TT'_i . When TT'_i is presented during the next “Get-out” process, the server cannot identify it since, during the inspection, it has never had access to it.

Note that if the same traveler is inspected again, the server will identify the *Tokens'* sequence. Nevertheless, this provides little additional information since both inspections can already be linked from the ID_{user} identity object the traveler has to present to prove their identity. The only additional information is that both journeys were made under the same e-ticket, which may be deduced anyway from their proximity in time. $\square$

Theorem 1 *The proposed system meets design objectives 1 and 2 (travelers cannot be automatically tracked).*

Proof Lemma 1 states that the server gets no information about the tokens created at the beginning of a journey when a ticket of the underlying system is validated. If the underlying ticket system is chosen to be both anonymous and unlinkable, e-ticket validations cannot be related among them.

Next, Lemma 2 ensures that tokens do not include information enabling their linkage and Lemma 3 states that no information enabling linkage is collected by the server when new tokens are activated. So during “Get-in” and “Get-out” operations, no information about the traveler nor information allowing token linkage is collected by the server.

Further, Lemma 4 states that the information collected by the inspector and the server do not allow linking the current traveler to other interactions during their journey, apart from the obvious conclusions derived from the identity object the traveler is required to show. $\square$

6.2 Security analysis against malicious travelers

The following section addresses the way in which the system protects the service provider from travelers aiming to breach the terms and conditions of tickets.

Lemma 5 *As a result of an execution of the ticket validation process, no more than M activated “In-travel” tokens can be obtained.*

Proof During the ticket validation process, the server computes exactly M blind signatures using its “In-travel” private key. Hence, no more than M activated (signed) “In-travel” tokens can result from this action. $\square$

Lemma 6 *When a set of tokens is linked to an identity object, all of them are linked to the same identity.*

Proof The tokens are linked to an ID_{user} object when the server blindly signs the *Tokens* sequence their HMAC digests have been included in. This sequence includes just one ID_{user} object. Hence, all the tokens are linked to the same identity. $\square$

Lemma 7 *During an inspection, the identity linked to a set of tokens cannot be modified.*

Proof An inspected traveler presents their current “In-travel” token to the inspector together with their *Tokens* sequence. After all the checkings, the traveler creates a new “In-travel” token whose HMAC digest replaces that of the inspected one in the *Tokens* sequence. The inspector checks that the remaining bits of the *Tokens* sequence, including those corresponding to the ID_{user} object, remain unmodified before signing the updated *Tokens'* sequence. $\square$

Theorem 2 *If a traveler validates an e-ticket linked to their ID_{user} object during the t -th time slot, and no other ticket is validated linked to the same identity object, then,*

1. *the traveler will perform M (single) journeys at most;*
2. *the traveler will perform the journeys before the conclusion of the $t + \ell$ time slot.*

Otherwise, they will incur the risk of being fined or their deposit token will be lost.

Proof In order not to be fined, a traveler inside the public transport system must be in possession of an activated “In-travel” token linked to their identity object, ID_{user} , via a non-expired *Tokens* sequence, and also linked to the current trip identifier, ID_{trip} , via a partially blind signature.

Each ID_{trip} identifier is unique, so an inspector will not accept it in a different transport system trip. Hence, a traveler who is about to begin a journey needs to link the correct ID_{trip} to their activated “In-travel” token. This binding is created through a partially blind signature computed by the server as long as a deposit token, which is then marked as no longer valid, is provided by the traveler. The traveler will receive a new free-of-charge deposit token as long as they run the get-out process at

the end of the journey. During this action, they have to provide an “In-travel” token which will be recorded as already spent.

Hence, performing a journey with a token linked to a proper ID_{trip} without losing the “deposit” token is not possible unless an “In-travel” token is provided and marked as no longer valid at the end. The validation of a ticket by a traveler will never enable the activation of more than M “In-travel” tokens (Lemma 5), which will all be linked to the same identity object (Lemma 6) being impossible to modify (Lemma 7). Consequently, no more than M journeys per validated ticket are possible.

As to caducity, a *Tokens* sequence is valid until the time slot committed by the traveler, which should be $t + \ell$, expires. The traveler gets no advantage from committing to a previous slot since, in that case, the *Tokens* sequence will expire before the deadline allowed by the transport system rules. If the commitment is to a posterior slot, namely $t + \ell + \delta$, the traveler will be fined if inspected before the $t + \delta$ slot. In this way, they are incurring some risk of being fined. $\square$

7 Experimental results

The actors considered in our experiments are: the *system server*, the *entry/exit stations* and the *mobile app (traveler)*. To simplify, we have considered the *entry/exit stations* to be part of the *system server*.

Prototypes have been implemented in Java. The *mobile app* has been developed specifically for the Android operating system. The `java.math.BigInteger` library has been used to implement cryptographic operations involving big integers.

Our implementation employs 2048 bits cryptography for RSA signatures and for all the operations involving hard-to-solve instances of the discrete logarithm problem. Hash digests have been computed by means of the SHA-256 function, while AES-256 has been used for symmetric encryption.

We have simulated the “Ticket validation” (described in Sect. 5.8 which calls the procedures from Sect. 5.6 and Sect. 5.7) and “Get-in” (described in Sect. 5.11 which calls the procedures from Sect. 5.9 and Sect. 5.10) procedures, which are the most relevant for the system. In fact, the “Get-out” procedure is mainly equivalent to the

Table 2 *Mobile app* running times (in milliseconds)

	Mobile phone		Ticket validation		Get-in
	Xiaomi model	GHz	Real-time	Pre-computed	
Times of Borges and Sebé	<i>Aquaris U Lite</i>	1.4	21528	8952	880
	<i>Aquaris U</i>	1.4	16414	4467	877
New proposal	<i>Redmi Note 9</i>	2.00	296	108	206
	<i>Redmi Note 9 S</i>	2.32	237	86	169

“Get-in” one, while the random inspection procedure is only run from time to time with no real-time constraints.

We have set parameters $M = 3$ (maximum number of journeys per ticket) and $l = 18$ (number of 5 min time slots embracing a 1 h 30 min validity period) in order to simulate a scenario equivalent to the experiments conducted by Borges and Seb  (2020).

Table 2 shows the average running times for the *mobile app*. Like Borges and Seb  (2020), the “Ticket validation” procedure has been simulated in two modes: *real-time* and *pre-computed*. In real-time mode, all operations are computed during the procedure execution. On the other hand, in the pre-computed mode, some operations are computed in advance and stored until the process execution. More precisely, the blinded values at step 2 of Sect. 5.8 were generated in advance. The “Get-in” procedure was only executed in real-time mode (it would get little advantage from parallelism).

We can observe that the running times obtained over Redmi devices are proportional to their CPU clock rate (GHz). The Redmi Note 9 S model has a faster CPU than the Redmi Note 9 model and achieves a 25% higher speed.

Although the clock rates of the mobile devices used for the present article are, on average, 50% higher than those of Borges and Seb  (2020), we observe that the most reduced running time of the present proposal for the “Ticket validation” procedure is 50 times smaller (86ms) than the fastest execution of Borges and Seb  (2020) (4467 ms). The fast execution of the current proposal is due to its design which does not include time-consuming cut-and-choose techniques as it happens in Borges and Seb  (2020). Regarding the “Get-in” procedure, we can observe the new proposal executions run up to five times faster.

Table 3 shows the average running times at the server part. Since the server cannot benefit from precomputations, only times in real-time mode are shown. On the server side, we include results both from serial and parallel implementations.

We observe that parallel executions benefit from a time reduction more or less proportional to the number of cores. In the most powerful machine (AMD Ryzen 7), a single server could participate in the validation of up to 2600 tickets

Table 3 System server running times (in milliseconds)

	System server			Ticket validation		Get-in	
	Processor	Cores	GHz	Serial	Parallel	Serial	Parallel
Times of Borges and Seb�	AMD Athlon	4	2.80	25867	7497	487	126
	Intel i7-6700	4	3.40	11812	2814	219	56
	Intel i7-8700	6	3.20	10103	1615	184	30
New proposal	AMD Athlon	4	2.80	834	218	744	213
	Intel i7-6700	4	3.40	675	150	648	149
	Intel i7-8700	6	3.20	291	51	279	52
	AMD Ryzen 7	8	3.70	196	23	183	22

per minute. Similar rates are obtained for the “Get-in” procedure where a single server is capable of processing 2720 traveler accesses per minute.

In the server case, we have been able to run experiments over the same server models (AMD Athlon, Intel i7-6700 and i7-8700) considered by Borges and Seb  (2020). We can observe that the “Get-in” procedure of the new proposal takes longer than in Borges and Seb  (2020), namely, running times have increased between 70% and 150%. This is due to the fact that during the “Next token activation” process, the server computes a number of exponentiations which is linear with parameter l . Nevertheless, an increase from 30 to 52 milliseconds (Intel i7-8700) still enables its execution with no perceptible delay for travelers. Regarding the “Ticket validation” procedure, its execution over the Intel i7-8700 model is 30 times faster than that of Borges and Seb  (2020). As it happened on the mobile app side, this is because of the avoidance of cut-and-choose checkings.

We would like to conclude this section by highlighting that the new proposal enables all the actions of the ticket system to be carried out in real time. Using up-to-date hardware, the “Ticket validation” process takes 86 and 23 ms at the mobile phone and the server, respectively. For the “Get-in” process (and also the “Get-out” one), the achieved running times are 169 (mobile phone) and 22 ms (server part).

8 Conclusion

This paper has presented a privacy-preserving electronic ticket system enabling the reusability of a ticket for a maximum number of journeys before an expiration time. Privacy and security against frauds are provided by using cryptography. The performance of the system was tested by means of a prototype, which enabled to state that the obtained running times make the system suitable for its deployment in a real scenario.

Future research needs to target the hardware aspects of the system. The proposal has been tested on up-to-date smartphones for the traveler side, which is a reasonable assumption. Nevertheless, its incorporation over smart card-based mobility systems is an open challenge which may require a non-trivial adaptation of the underlying cryptography. Regarding the service-provider side, public transport entry/exit stations are already automated devices, but the proposed system requires communication and processing capacities far beyond those of devices used just for the reading of ticket numbers via near field communication (NFC). For this reason, the upgrading cost for such devices would have to be investigated in detail.

Acknowledgements The Cryptography and Graphs Research Group has been recognized as a consolidated research group 2021-SGR-00434 by Generalitat de Catalunya.

Author Contributions All authors contributed to the study conception and system design. Francesc Seb  made the formal security analysis. Ricard Borges implemented the prototype and conducted the experiments. The first draft of the manuscript was written by Francesc Seb  and all authors commented on previous versions of the manuscript. All authors read and approved the final manuscript.

Funding Open Access funding provided thanks to the CRUE-CSIC agreement with Springer Nature. This paper is part of the R&D+i project PID2021-124613OB-I00 funded by MICIU/AEI/10.13039/501100011033 and FEDER,EU.

Declarations

Conflict of interest. The authors have no conflict of interest to declare that are relevant to the content of this article.

Code availability Source code used for experiments is available from the authors upon request.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Abe M, Okamoto T (2000) Provably secure partially blind signatures. In: Bellare M (ed) *Advances in cryptography—CRYPTO 2000*. Springer, Berlin, Heidelberg, pp 271–286. https://doi.org/10.1007/3-540-44598-6_17
- Aboud SJ, Al-Fayoumi M, Al-Fayoumi M (2017) Efficient e-tickets fare scheme for time-typed services. *J Internet Bank Commerce* 22(1):1–20
- Barabino B, Lai C, Olivo A (2020) Fare evasion in public transport systems: a review of the literature. *Public Transport* 12:27–88. <https://doi.org/10.1007/s12469-019-00225-w>
- Bellare M, Canetti R, Krawczyk H (1996) Keying hash functions for message authentication. In: Koblitz N (ed) *Advances in cryptography—CRYPTO '96*. Springer, Berlin, Heidelberg, pp 1–15. https://doi.org/10.1007/3-540-68697-5_1
- Benarroch D, Campanelli M, Fiore D, Gurkan K, Kolonelos D (2021) Zero-knowledge proofs for set membership: efficient, succinct, modular. In: Borisov N, Diaz C (eds) *Financial cryptography and data security*. Springer, Berlin, pp 393–414. <https://doi.org/10.1007/s10623-023-01245-1>
- Borges R, Seb  F (2020) A construction for providing reusability to mobile phone-based e-tickets. *IEEE Access* 8:101386–101397. <https://doi.org/10.1109/ACCESS.2020.2998504>
- Chaum D (1984) Blind signature system. In: Chaum D (ed) *Advances in cryptography—CRYPTO '83*. Springer, Boston, MA, pp 153–153. https://doi.org/10.1007/978-1-4684-4730-9_14
- Chen L, Moody D, Regenscheid A, Robinson A (2023) Digital signature standard (DSS). Tech. Rep. FIPS 186-5, Federal Inf. Process. Stds. (NIST FIPS), National Institute of Standards and Technology, Gaithersburg, MD. <https://doi.org/10.6028/NIST.FIPS.186-5>
- Covic F, Vo  S (2019) Interoperable smart card data management in public mass transit. *Public Transport* 11:523–548. <https://doi.org/10.1007/s12469-019-00216-x>
- Elgamal T (1985) A public key cryptosystem and a signature scheme based on discrete logarithms. *IEEE Trans Inf Theory* 31(4):469–472. <https://doi.org/10.1109/TIT.1985.1057074>
- European Commission (2016) Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation) (Text with EEA relevance). URL <https://eur-lex.europa.eu/eli/reg/2016/679/oj>
- Han J, Chen L, Schneider S, Treharne H, Wesemeyer S (2021) Privacy-preserving electronic ticket scheme with attribute-based credentials. *IEEE Trans Depend Secure Comput* 18(4):1836–1849. <https://doi.org/10.1109/TDSC.2019.2940946>

- Heydt-Benjamin TS, Chae HJ, Defend B, Fu K (2006) Privacy for public transportation. In: Danezis G, Golle P (eds) Privacy enhancing technologies. Springer, Berlin, pp 1–19. https://doi.org/10.1007/11957454_1
- Lei Y, Quintero A, Pierre S (2009) Mobile services access and payment through reusable tickets. *Comput Commun* 32(4):602–610. <https://doi.org/10.1016/j.comcom.2008.11.035>
- Miller FP, Vandome AF, McBrewster J (2009) Advanced encryption standard. Alpha Press
- Milutinovic M, Decroix K, Naessens V, De Decker B (2015) Privacy-preserving public transport ticketing system. In: Samarati P (ed) Data and applications security and privacy XXIX. Springer, Cham, pp 135–150. https://doi.org/10.1007/978-3-319-20810-7_9
- Mut-Puigserver M, Payeras-Capellà MM, Ferrer-Gomila JL, Vives-Guasch A, Castellà-Roca J (2012) A survey of electronic ticketing applied to transport. *Comput Secur* 31(8):925–939. <https://doi.org/10.1016/j.cose.2012.07.004>
- Naor M, Yung M (1989) Universal one-way hash functions and their cryptographic applications. In: Proceedings of the twenty-first annual ACM symposium on theory of computing. Association for Computing Machinery, New York, NY, USA, STOC '89, pp 33–43. <https://doi.org/10.1145/73007.73011>
- Quercia D, Hailes S (2005) MOTET: Mobile transactions using electronic tickets. In: First International Conference on security and privacy for emerging areas in communications networks, pp 374–383. <https://doi.org/10.1109/SECURECOMM.2005.27>
- Rantasila K (2015) The impact of mobility as a service concept to land use in Finnish context. In: 2015 International Conference on sustainable mobility applications, renewables and technology, pp 1–7. <https://doi.org/10.1109/SMART.2015.7399229>
- Sadeghi AR, Visconti I, Wachsmann C (2008) User privacy in transport systems based on RFID e-tickets. In: PiLBA'08 Privacy in Location-Based Applications, pp 102–121
- Vives-Guasch A, Payeras-Capellà M, Mut-Puigserver M, Castellà-Roca J (2011) E-ticketing scheme for mobile devices with exculpability. In: Garcia-Alfaro J, Navarro-Arribas G, Cavalli A, Leneutre J (eds) Data privacy management and autonomous spontaneous security. Springer, Berlin, Heidelberg, pp 79–92. https://doi.org/10.1007/978-3-642-19348-4_7
- Vives-Guasch A, Payeras-Capellà MM, Mut-Puigserver M, Castellà-Roca J, Ferrer-Gomila JL (2012) A secure e-ticketing scheme for mobile devices with near field communication (NFC) that includes exculpability and reusability. *IEICE Trans Inform Syst E* 95.D(1):78–93. <https://doi.org/10.1587/transinf.E95.D.78>
- Zhan Y, Yuan F, Shi R, Shi G, Dong C (2024) PriTKT: a blockchain-enhanced privacy-preserving electronic ticket system for IoT devices. *Sensors* 24(2):496. <https://doi.org/10.3390/s24020496>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.